

Desafío 1:

Se recorre la cola sacando cada elemento que se guardará en una pila para invertir el orden y en una cola auxiliar para conservar el orden original. Después se comparan los elementos y si alguno es distinto, la cola no es palíndromo. Finalmente, se reconstruye la cola original.

```
boolean esPalindromo(ColaTDA c) {
    PilaTDA p = new PilaTDA(); p.InicializarPila();
    ColaTDA aux = new ColaTDA(); aux.InicializarCola();

    while (!c.ColaVacia()) {
        int x = c.Primer(); c.Desacolar();
        p.Apilar(x); aux.Acolar(x);
    }

    while (!aux.ColaVacia()) {
        int a = aux.Primer(); aux.Desacolar();
        int b = p.Tope(); p.Desapilar();
        if (a != b) return false;
        c.Acolar(a);
    }

    return true;
}
```

Desafío 2:

Se recorre la cola desacolando todos los elementos, se cuentan los de prioridad 1 y se guardan en auxiliares. Luego se reconstruye la cola original.

```
int contarP1(ColaPrioTDA cp) {
    ColaPrioTDA a1 = new ColaPrioTDA(); a1.InicializarCola();
    ColaPrioTDA a2 = new ColaPrioTDA(); a2.InicializarCola();
    int c = 0;

    while (!cp.ColaVacia()) {
        int x = cp.Primer(), p = cp.Prioridad();
        cp.Desacolar();
        if (p == 1) c = c + 1;
        a1.AcolarPrioridad(x, p);
    }

    while (!a1.ColaVacia()) {
        int x = a1.Primer(), p = a1.Prioridad();
```

```

        a1.Desacolar();
        cp.AcolarPrioridad(x, p);
        a2.AcolarPrioridad(x, p);
    }

    while (!a2.ColaVacia()) a2.Desacolar();

    return c;
}

```

Desafío 3:

Se utilizaron dos estructuras auxiliares: un Conjunto y una Pila. Se fue desapilando la Pila original y por cada elemento se chequeó con el Conjunto si ya había aparecido antes. Si ya estaba, la Pila tiene repetidos. Si no, se lo agregó al Conjunto y se lo guardó en la Pila auxiliar. Al final, se restauró la Pila original desapilando la auxiliar.

```

public static boolean PilaUnica(PilaTDA p) {
    ConjuntoTDA c = new ConjuntoTDA();
    PilaTDA aux = new PilaTDA();
    c.InicializarConjunto();
    aux.InicializarPila();

    boolean unica = true;

    while (!p.PilaVacia()) {
        int elem = p.Tope();
        p.Desapilar();

        if (c.Pertenece(elem)) {
            unica = false;
        }
        c.Agregar(elem);
        aux.Apilar(elem);
    }

    while (!aux.PilaVacia()) {
        int elem = aux.Tope();
        aux.Desapilar();
        p.Apilar(elem);
    }

    return unica;
}

```

Desafío 4:

Es necesario recorrer dos veces y utilizar conjuntos auxiliares para evitar la pérdida de datos de los conjuntos originales

```
public static void diferenciaSimetrica(ConjuntoTA A, ConjuntoTA B, ConjuntoTA C) {
    ConjuntoTA auxA = new ConjuntoDinamico();
    ConjuntoTA auxB = new ConjuntoDinamico();

    auxA.inicializarConjunto();
    auxB.inicializarConjunto();
    C.inicializarConjunto();

    while (!A.conjuntoVacio()) {
        int x = A.elegir();
        A.sacar(x);

        if (!B.pertenece(x)) {
            C.agregar(x);
        }

        auxA.agregar(x);
    }

    while (!auxA.conjuntoVacio()) {
        int x = auxA.elegir();
        auxA.sacar(x);
        A.agregar(x);
    }
    while (!B.conjuntoVacio()) {
        int x = B.elegir();
        B.sacar(x);

        if (!A.pertenece(x)) {
            C.agregar(x);
        }

        auxB.agregar(x);
    }
    while (!auxB.conjuntoVacio()) {
        int x = auxB.elegir();
        auxB.sacar(x);
        B.agregar(x);
    }
}
```

}

Desafío 5:

1. ¿Qué elemento sale primero al Desacolar?
 - El '5' , porque tiene $P=2$ (mayor prioridad) y fue el primero en entrar con esa prioridad.
2. ¿Qué elemento sale último?
 - El '-4' , porque tiene $P=0$ (menor prioridad) y fue el último en entrar con esa prioridad.
3. Dentro de la misma prioridad, ¿en qué orden salen?
 - En orden FIFO, el mismo orden en que entraron.
4. Si hubiera DOS ceros, ¿ambos entrarían en $P=1$?
 - Ambos ceros entrarían en $P=1$, ya que AcolarPrioridad no verifica unicidad, simplemente acola el elemento con su prioridad. Saldrían en orden FIFO entre ellos.